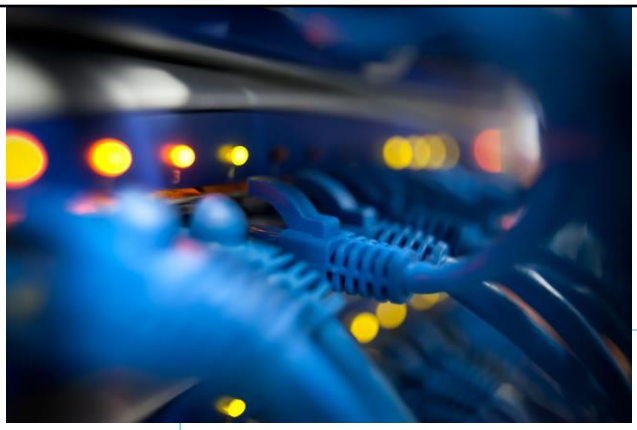
Back-End Development

Web Servers,
Back-end Abstractions,
Building APIs

1



Lec 10: Web Servers

What is a web server?

A **web server** is both a type of software and a piece of hardware that **stores, processes, and delivers** web pages to users over the internet.

Steps:

- User types an address "www.example.com," or click a link
- Computer (called the "client") sends a request to the web server
- Server then sends the right information back to the client.

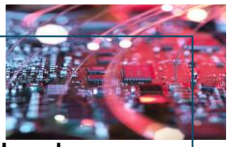
- A **web server** is both a type of software and a piece of hardware that stores, processes, and delivers web pages to users over the internet. When you type in a website address, like "www.example.com," or click a link, your computer (called the "client") sends a request to the web server, which then sends the right information back to your computer.
- Think of a web server as a **waiter in a restaurant**:
- When you, the customer, ask for a dish (like clicking a link), the waiter (the web server) takes your order, goes to the kitchen, and brings you your food (in this case, the web page).
- The main job of the server is to listen, understand, and respond to requests from clients (like a browser on your phone or computer).

Relatable Example

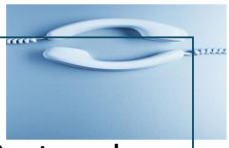
- Think of a web server as a **waiter in a restaurant**:
- When you, the customer, ask for a dish (like clicking a link), the waiter (the web server) takes your order, goes to the kitchen, and brings you your food (in this case, the web page).
- The main job of the server is to listen, understand, and respond to requests from clients (like a browser on your phone or computer).

Components of a Web Server

Software



Hardware



Protocols

Facilitate intercommunication between web servers and computers. Examples are: **HTTPS** for static content delivery such as images and documents; **SMTP** for mail delivery; and **FTP** for file transfer and storage.

A web server's software component uses HTTP to handle users' requests and responses on a website.

The hardware consists of computers that store website files and documents and the web server's software; such files include HTML, CSS, and JavaScript files. The hardware enables internet communication to share these files. Examples of web server hardware include CPU, RAM, GPU, and SSDs.

Protocols facilitate intercommunication between web servers and computers. They include HTTPS, which helps in static content delivery such as images and documents; SMTP, which allows mail delivery; and FTP, which helps in file transfer and storage.

Web Server -Hardware

A computer ("server") connected to the internet (or any network) [Physical Entity]

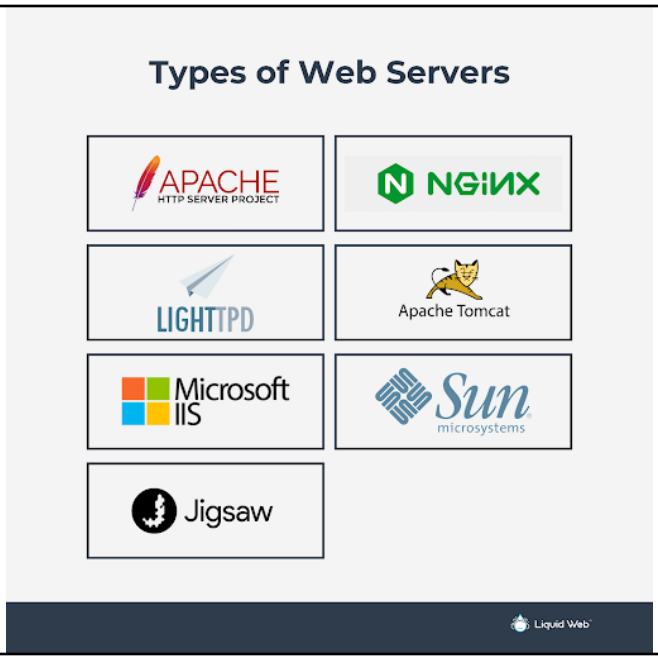
Properties of contemporary web servers

- Part of large datacenters
- Latency is geographically dependent, so web servers are often geographically distributed (works through, e.g., DNS)
- **Virtual servers:** Physical servers can host many virtualized (web) servers

Can also be your own computer (localhost)

Key Concepts- Revision

- **Requests and Responses**: Every time you type a URL or click on a link, your browser sends a request to the web server. The server then responds with the webpage or file you wanted.
- **HTTP Protocol**: Web servers and clients communicate through the **HTTP** protocol, which stands for **Hypertext Transfer Protocol**. This protocol tells the web server how to handle and deliver the files requested.
- **Serving Content**: Web servers deliver different types of files, like:
  - HTML files (the code for web pages),
  - Images (like PNG, JPG),
  - Styles (CSS), and
  - Scripts (JavaScript files).



Various web servers with numerous software solutions are available to meet the diverse needs of websites and web applications. The following are common examples.

Apache HTTP server

Apache HTTP Server is an open-source software built for operating systems like Windows, UNIX, and Mac OS. It provides secure and standard web server software for websites.

As a versatile web server software, Apache's architecture provides a customizable functionality that creates a wide range of features. Among other web servers, Apache has about 20.7 percent of the market. Due to its popularity, it's the go-to web server for developers, web owners, and hosting service providers.

NGINX

NGINX is another popular web server that manages 23.21 percent of all websites. It's widely used for caching, load balancing, media streaming, etc. NGINX offers low memory consumption and a high asynchronous mode of serving web requests.

Internet information services (IIS)

Microsoft developed and maintained the IIS web server. It provides a secure website layer with a low surface area and maximum functionality.

Sun Java system

This multithreaded web server software provides high performance and scalability to

websites. It operates multiple single-threaded child processes simultaneously and distributes all incoming requests among the child processes for better performance. Jigsaw server, implemented on the JAVA programming language, is an open-source web server software with a modular architecture. It uses an object-oriented approach to store files and process requests. Sun Java manages about 0.1 percent of all websites.

Apache Tomcat

Apache Tomcat is an open-source Java Server Page container and Java servlet used to host and run Java applications and websites. It helps Java apps with dynamic content for multiple requests. It's an intermediary between the web server and the Java application to ensure efficient and secure data access.

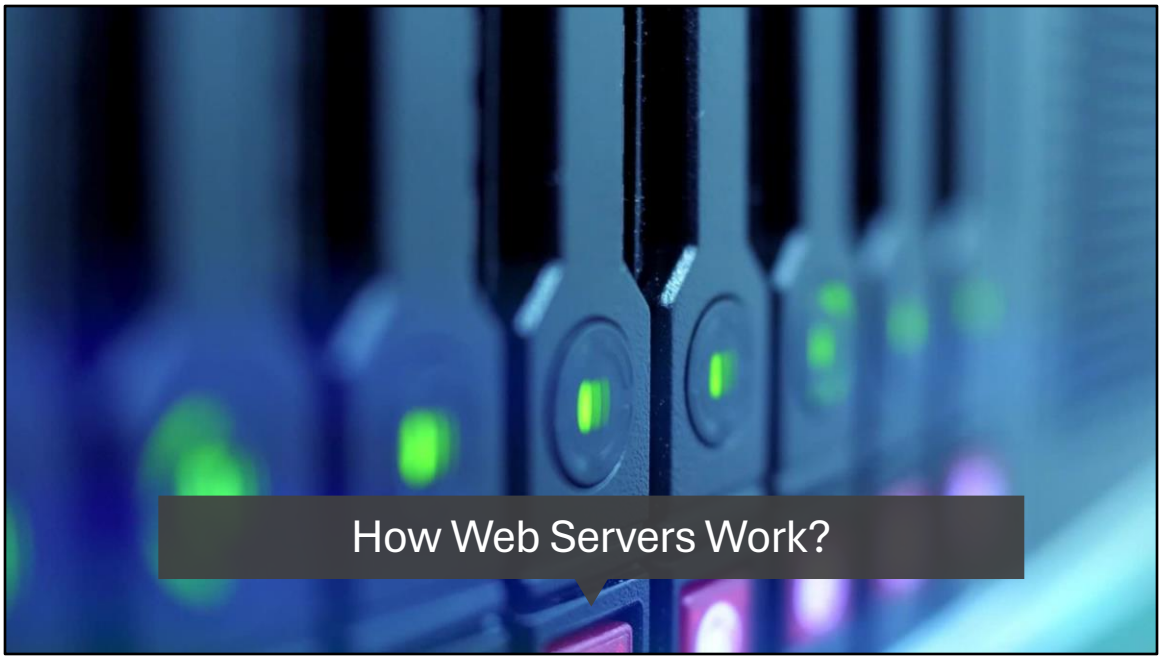
Regardless of the chosen web server software, its architecture is crucial in determining its performance and scalability.

Server Side Code

- Server side code could be simply a file that the server passes back to the client, which is then displayed for the user

OR

- Could be a program, written in PHP, Perl, Ruby, Python, C, or some other language that performs an operation and sends results back to the client browser.


How Web Servers Work?

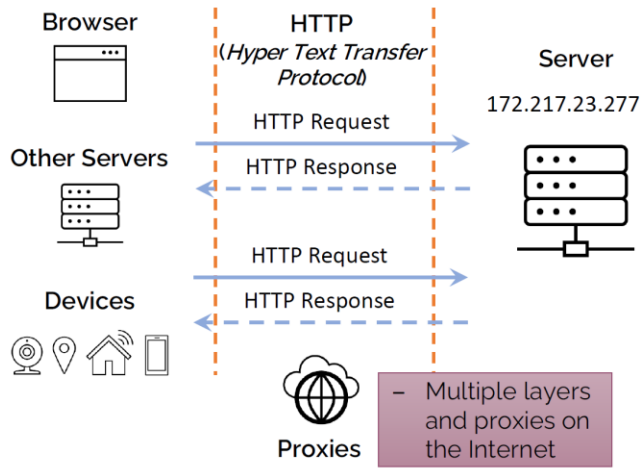
High Level Web Overview

- Servers wait for requests
- They serve web resources

Client https://www.google.com

http://www.google.com 172.217.23.277

- **Domain Name System (DNS)**
  - translating hostname to IP address



- Multiple layers and proxies on the Internet

Client-Server Relationship

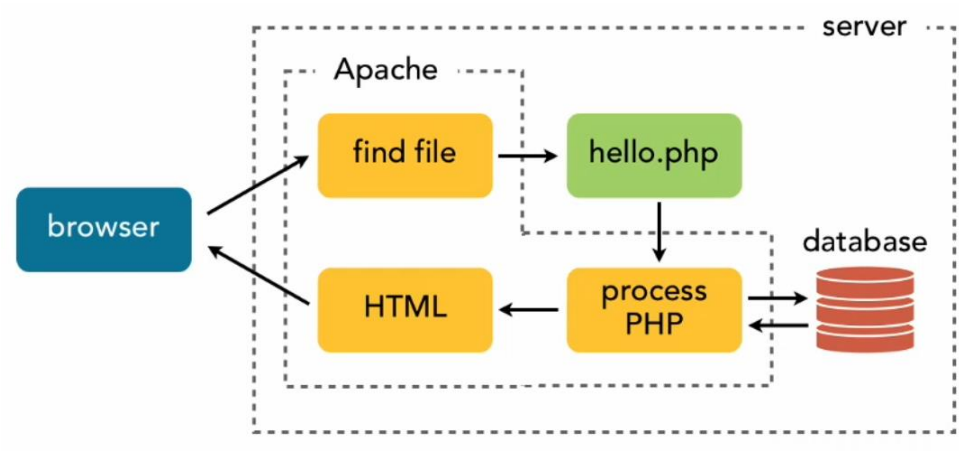
- **Client**: This is usually a web browser, like Chrome, Firefox, or Safari. The client asks for information from the web server.
- **Server**: This is the machine that holds the data or files the client wants. When a client makes a request, the server responds.
- For example, "www.example.com":
- The **browser** sends a request to the server where "example.com" is hosted.
- The **server** receives this request, finds the right file (like an HTML page), and sends it back.
- The **browser** then displays the page.

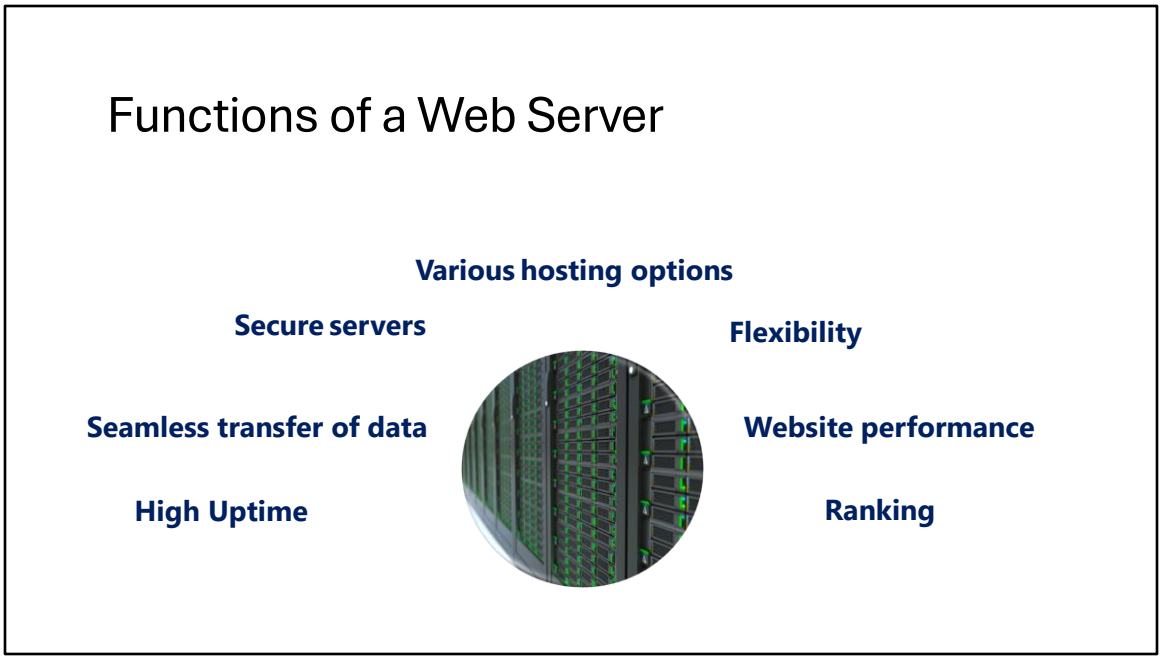
The Request-Response Cycle

- Every time you visit a webpage, your browser and the web server go through something called the **request-response cycle**. Here's what happens:
- **Request**: The browser sends a request for the webpage. This request is typically a URL, which points to a particular resource (like an HTML file) on the server.
- **Processing**: The web server reads the request and decides how to respond. It finds the necessary files or scripts needed to fulfill the request.
- **Response**: The server sends back the requested data (HTML, images, or other files) to the client.
- **Rendering**: The client (browser) receives the data and renders (displays) the webpage.

Key-Components

- **HTTP Protocol**: The primary protocol that governs how the web server and the client communicate. It includes requests like GET (to fetch data) and POST (to send data).
- **TCP/IP Stack**: This is the foundation of data transfer over the internet. It's responsible for ensuring that the data packets between the client and server are transmitted accurately.
- **Real-life Example**
- Think of visiting a library:
- You (the client) ask the librarian (the web server) for a book.
- The librarian finds the book and hands it to you.
- In the same way, the client requests data, and the server locates it and delivers it back to the client.


server
Apache
find file
hello.php
browser
database
HTML
process
PHP



High uptime

A web server's primary function is to provide a standby hardware infrastructure to store websites and maintain high uptime so that users can access hosted websites and web applications.

Seamless transfer of data

A web server provides an adequate and safe delivery of online content.

Secure servers

Web server hosts can provide robust security protocols and reduce cyber attacks, including Secure Sockets Layer (SSL),Secure Sockets Shell (SSH), and reverse proxy.

Various hosting options

Web servers provide different hosting options to accommodate diverse needs and requirements. These options include shared hosting, VPS hosting (virtual private servers), dedicated servers, and cloud hosting, each with features and resources.

Flexibility

Web servers offer high flexibility and support for programming languages, databases, and frameworks. This flexibility allows developers to choose the technologies that best suit their project requirements.

Website performance

Web servers enhance a website's performance by clearing the storage cache of files

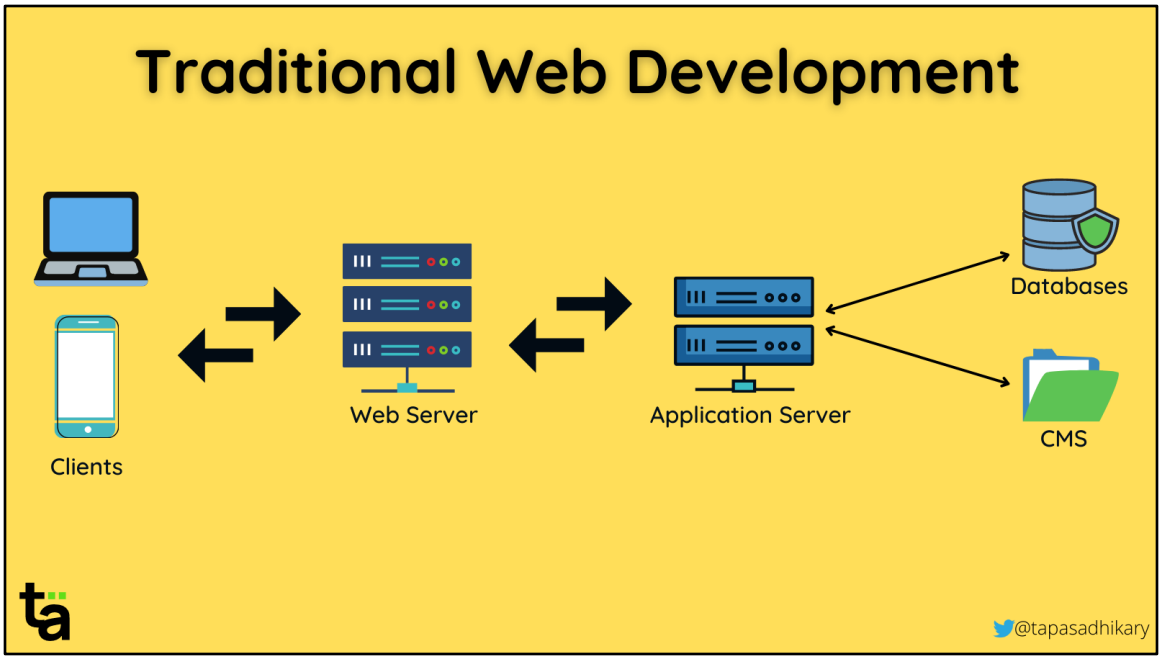
that have no use. Caches take up space on a web server and slow down a website. Clearing the cache provides more storage space for other valuable files and documents.

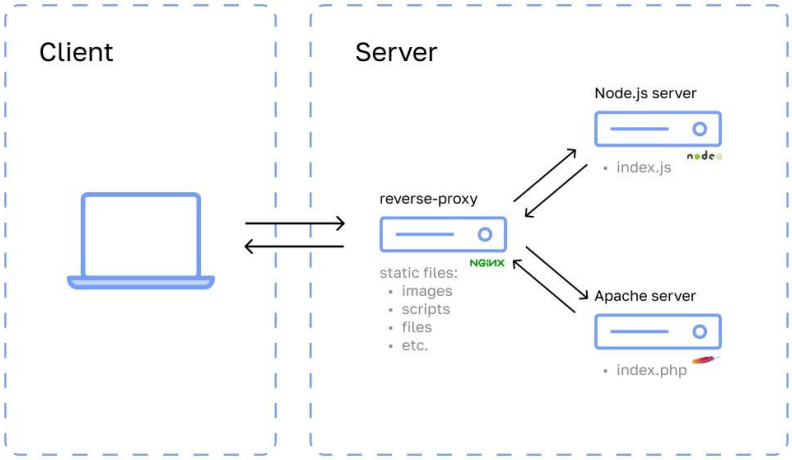
Ranking

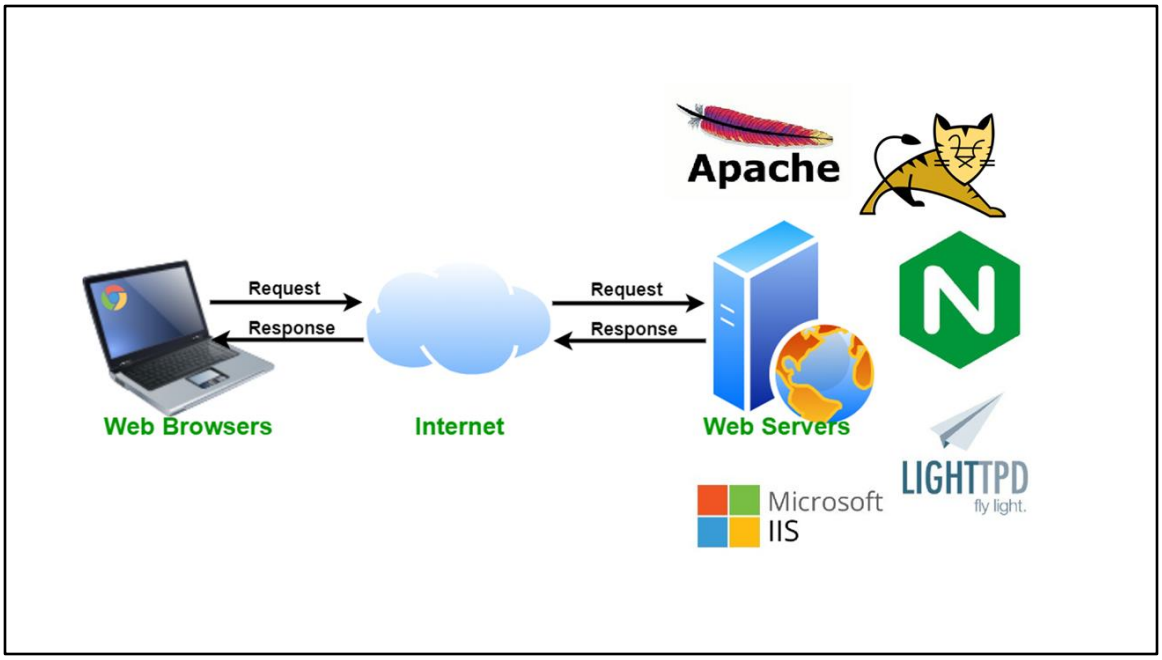
Web servers improve website ranking by optimizing page load and server response times. Faster page speeds and server response times contribute to better user experiences, a significant ranking factor for search engines like Google.

Types of Web Servers

Web servers can be **categorized** based on how they work and what they're capable of delivering. We wil ldiscuss two main types:


Traditional Web Development
Clients
Web Server
Application Server
Databases
CMS
@tapasadhikary


Client
Server
Node.js server
• index.js
reverse-proxy
static files:
• images
• scripts
• files
• etc.
Apache server
• index.php



1. Apache HTTP Server (Apache)

What is Apache?

Apache HTTP Server, commonly referred to simply as **Apache**, is one of the oldest and most widely used open-source web servers. Apache was developed by the **Apache Software Foundation** and has been in use since 1995. It is known for its flexibility, configurability, and ability to run on various operating systems like Linux, Unix, macOS, and Windows.

Key Features:

•**Modular Architecture**: Apache allows the addition of various modules (e.g., mod_php for PHP, mod_ssl for SSL support, mod_rewrite for URL rewriting). This makes Apache extremely versatile.

•**Dynamic Content**: Apache is particularly strong at handling dynamic content using server-side scripting modules (e.g., mod_php for PHP, mod_python for Python).

•**.htaccess Files**: Apache can use .htaccess files to configure settings at the directory level, which is especially useful for shared hosting environments.

•**Flexibility**: Apache is flexible and configurable, which is why it's often used for a wide variety of web applications and websites.

Use Cases:

•Hosting websites with dynamic content (e.g., WordPress, LAMP stack).

•Shared hosting environments, where .htaccess and per-directory configuration are important.
•Websites that require deep customization of server behavior through configuration files.

Platform Support:

•Cross-platform (Linux, Unix, macOS, Windows).

Performance:

•Apache tends to use more memory and CPU under heavy load due to its process/thread-based architecture. However, it can be optimized with different multi-processing modules like **mpm_event** or **mpm_worker**.

2. Nginx

What is Nginx?

Nginx (pronounced "engine-x") is an open-source, high-performance web server designed for speed and scalability. It was originally built to handle **high concurrency** and **low resource usage**, and it has quickly gained popularity, particularly for modern web applications.

Key Features:

•**Event-Driven, Asynchronous Model**: Nginx uses an event-driven model to handle incoming requests. It can process many requests at once without the need for creating new threads or processes for each request, making it more efficient than Apache under high traffic.
•**Reverse Proxy and Load Balancing**: Nginx is commonly used as a reverse proxy server and load balancer in front of application servers, distributing traffic across multiple backend servers.
•**Static Content**: Nginx is optimized for serving static content quickly and with low memory usage. This makes it ideal for serving static files like HTML, CSS, JavaScript, images, and videos.
•**SSL Termination**: Nginx is often used to handle SSL/TLS encryption (SSL termination) efficiently.

Use Cases:

•**Reverse Proxy Server**: Commonly used to reverse proxy traffic to backend servers (e.g., Node.js, PHP-FPM, or application servers).
•**Static Content Serving**: Great for serving static content, which is why it's used in many high-traffic environments.
•**High-Concurrency Environments**: Nginx is optimized for environments where handling many concurrent connections is necessary (e.g., media streaming, content-heavy websites).

Platform Support:

•Cross-platform (Linux, Unix, macOS, Windows). However, it's most commonly used on Linux-based systems.

Performance:

•**Very fast and efficient**, particularly for static content and reverse proxy use cases.

Due to its asynchronous, event-driven model, Nginx is highly scalable and handles many concurrent requests with minimal memory and CPU usage.

3. Microsoft IIS (Internet Information Services)

What is IIS?

Microsoft IIS (Internet Information Services) is a **proprietary web server** developed by Microsoft for Windows Server operating systems. It is closely integrated with the Windows ecosystem, making it a popular choice for enterprises that use Microsoft technologies.

Key Features:

•**Integrated with Windows**: IIS is tightly integrated into the Windows Server environment, making it a natural choice for organizations that use other Microsoft products (e.g., .NET applications, Windows authentication, SQL Server).

•**GUI Management Interface**: IIS provides a user-friendly **graphical user interface (GUI)** for configuration and management, making it easier for non-technical users to administer the server.

•**Security Features**: IIS has strong security features, including request filtering, URL authorization, and support for Windows Authentication and Active Directory integration.

•**Support for .NET**: IIS is optimized for serving **ASP.NET applications** and integrates well with the **Microsoft stack** (e.g., SQL Server, Active Directory).

Use Cases:

•**ASP.NET Web Applications**: IIS is the go-to web server for serving ASP.NET applications (MVC, Web API, etc.).

•**Microsoft Ecosystem**: If your environment is heavily tied to Microsoft technologies like Windows Server, SQL Server, or Active Directory, IIS is a natural choice.

•**Enterprise Environments**: IIS is often chosen by large organizations due to its integration with Windows, security features, and GUI management.

Platform Support:

•Windows only (Windows Server and Windows OS).

Performance:

•IIS is quite efficient in a Windows environment, especially for **Microsoft-based applications**. However, it's not as efficient as Nginx for serving static content or as flexible as Apache for handling dynamic content across multiple platforms.

Apache vs Nginx

- **Apache** is best suited for **dynamic content** serving and provides a highly customizable environment with powerful modules for various back-end technologies.
- **Nginx** excels at handling **high-traffic environments** where **static content** and **load balancing** are essential. It is known for its efficiency and low resource usage, making it ideal for serving static files and as a reverse proxy.
- In real-world deployments, many organizations use **both** Apache and Nginx together
- **Nginx** acts as a reverse proxy in front of **Apache** (or other backend servers) to handle incoming requests, serve static content, and distribute the load, while Apache is used for handling dynamic content. This is a common setup for high-performance websites and applications.

Apache and **Nginx** are two of the most popular web servers used to serve web content (like websites, web applications, and APIs) over the internet. They each have their own unique features, benefits, and trade-offs. Let's explore each of them in detail:

1. Apache HTTP Server (Apache)

What is Apache?

Apache, or **Apache HTTP Server**, is one of the oldest and most widely used web servers. It's open-source software developed by the **Apache Software Foundation**. Apache serves web content, handles HTTP requests, and can serve static content (like HTML, CSS, JavaScript, images) and dynamic content (like PHP, Python, etc.) using different modules.

Key Features of Apache:

•**Modular Architecture**: Apache is highly modular, meaning you can extend its functionality with various modules (e.g., mod_php for PHP, mod_ssl for SSL/TLS support).

•**Dynamic Content Handling**: Apache is great for serving dynamic content using modules like mod_php (for PHP), mod_python, mod_wsgi, etc. This makes it ideal for websites and applications that require server-side scripting.

•**.htaccess Files**: Apache allows for per-directory configuration using .htaccess files,

which enable fine-grained control over settings on a directory or file basis (e.g., redirects, security settings, URL rewrites).
•**Multi-threaded**: Apache uses a multi-threaded or multi-process model depending on its configuration, meaning it can handle multiple requests at the same time.
Use Cases of Apache:
•Websites and applications requiring dynamic content generation.
•LAMP stack (Linux, Apache, MySQL, PHP) applications, especially for legacy applications.
•Hosting on shared servers where customization per directory is needed using .htaccess.
Drawbacks of Apache:
•**Performance**: Apache is often slower than Nginx, especially under high traffic loads, because it creates a new thread or process for each request (though this can be mitigated with optimizations).
•**Memory Usage**: Apache can be more memory-intensive due to its process-based model.
2. Nginx
What is Nginx?
Nginx (pronounced "engine-x") is an open-source web server and reverse proxy server that is designed to be high-performance, highly scalable, and efficient. It was originally created by **Igor Sysoev** in 2004 and has become widely popular due to its ability to handle large amounts of concurrent traffic with minimal resource consumption.
Nginx is commonly used as a **reverse proxy server**, meaning it forwards client requests to other servers (like application servers or databases), but it can also be used to serve static content.
Key Features of Nginx:
•**Event-driven Architecture**: Nginx uses an event-driven, asynchronous model. This allows it to handle many requests simultaneously with a small amount of memory and without spawning new threads or processes. This makes Nginx much more scalable and efficient than Apache.
•**Reverse Proxying**: Nginx is often used as a reverse proxy server to balance the load between multiple application servers. It efficiently routes incoming requests to the appropriate backend servers and can distribute the traffic.
•**Handling Static Content**: Nginx is known for its **superb handling of static content**. It is faster than Apache when serving static files like HTML, CSS, JavaScript, images, and videos due to its low overhead.
•**SSL/TLS Termination**: Nginx can offload SSL/TLS processing (SSL termination), reducing the burden on backend servers.
•**High Performance & Low Resource Usage**: Nginx is highly efficient and uses far fewer resources (memory and CPU) when handling large numbers of requests compared to Apache.

Use Cases of Nginx:

•**Reverse Proxy and Load Balancing**: Nginx is widely used in production environments as a reverse proxy and load balancer for distributing traffic across multiple servers.

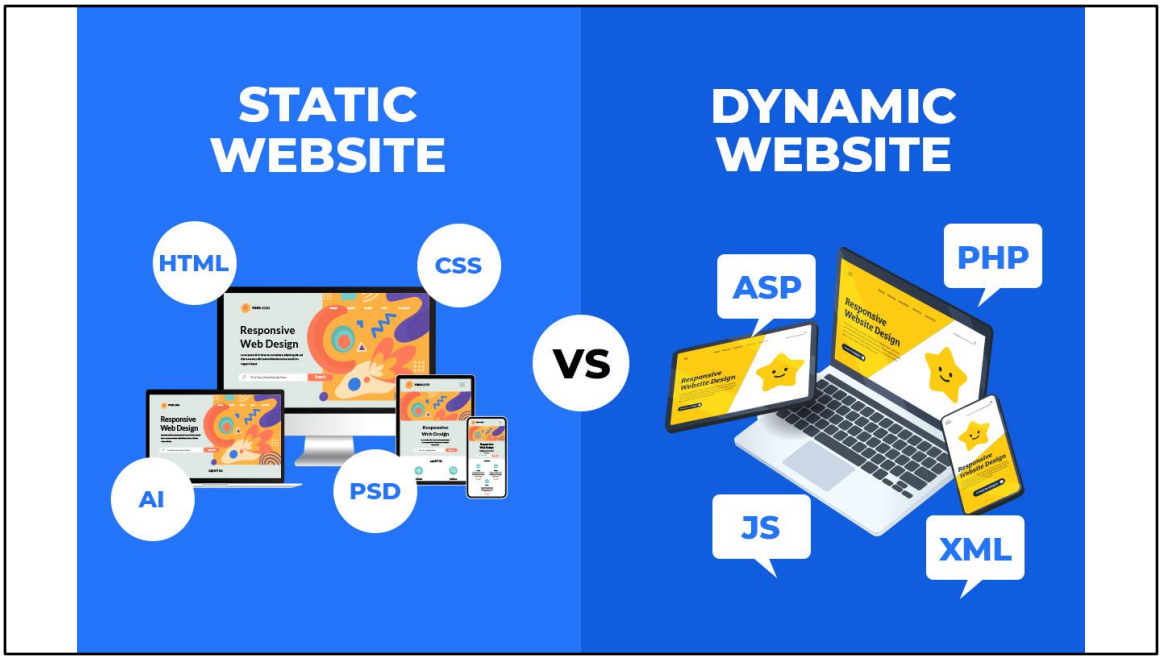
•**Serving Static Content**: Nginx is ideal for serving static files like HTML, CSS, images, etc.

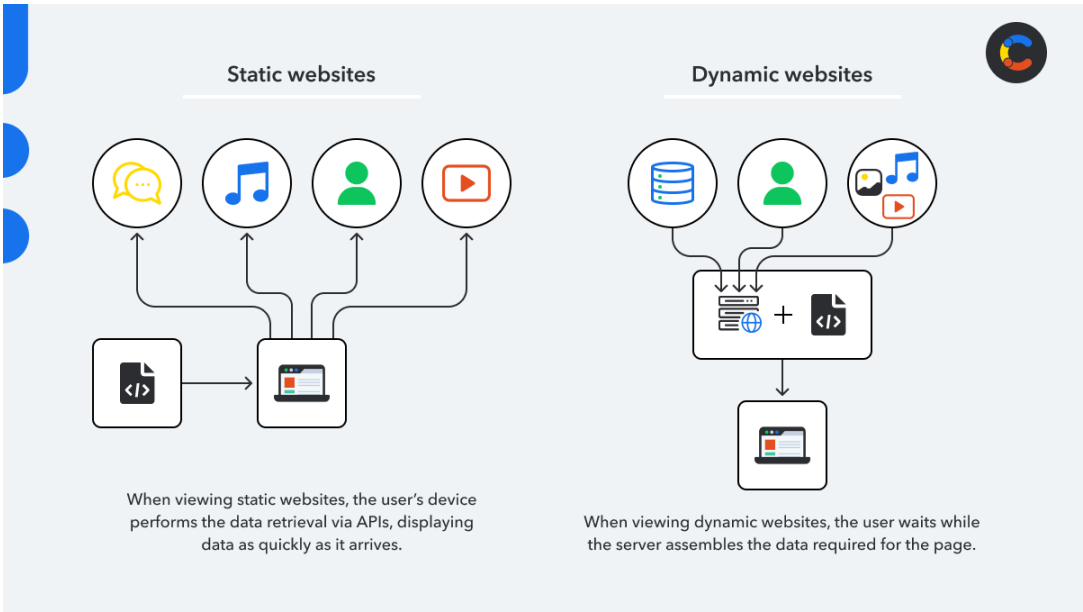
•**Scaling Web Applications**: It's often used as a gateway server in front of application servers (e.g., Node.js, PHP-FPM) for better performance and handling high traffic.

Drawbacks of Nginx:

•**Dynamic Content Handling**: While Nginx can handle dynamic content via integration with other backends (e.g., FastCGI, PHP-FPM, or application servers), it's not as native as Apache's integration with dynamic content. This can be a drawback if you're heavily using server-side scripting.

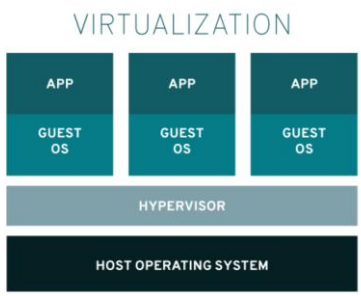
•**Configuration Complexity**: Nginx configuration can sometimes be harder for beginners to understand, especially in terms of advanced features like load balancing or reverse proxying.


STATIC WEBSITE
HTML
CSS
AI
PSD
VS
DYNAMIC WEBSITE
ASP
PHP
JS
XML

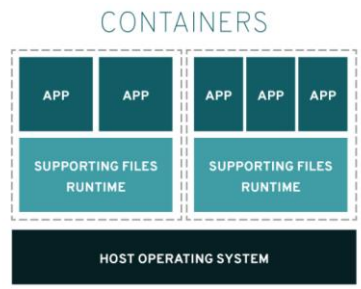

Static websites
Dynamic websites
When viewing static websites, the user's device performs the data retrieval via APIs, displaying data as quickly as it arrives.
When viewing dynamic websites, the user waits while the server assembles the data required for the page.

Virtual Servers and Containers

One physical server can host multiple **virtual servers** and/or **containers**



- Enables multiple virtual instances of different OSs to run in isolation through "hypervisor"
- Hypervisors divide physical resources so that virtual servers can use them and "translates" kernel operations



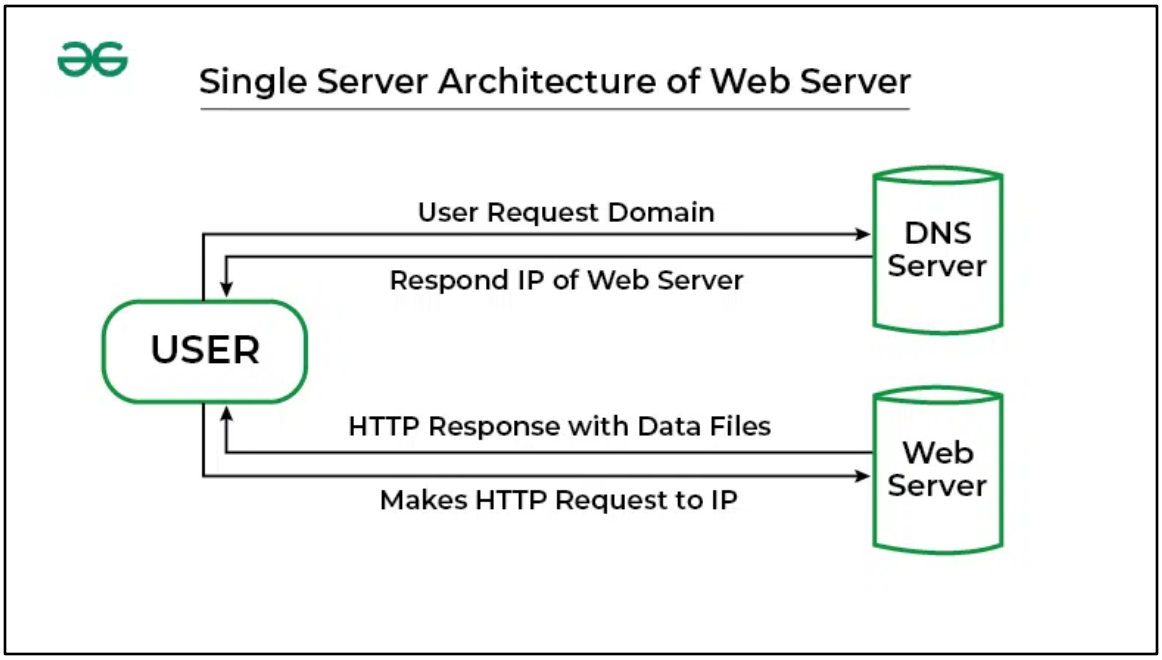
- "feel" like virtual machines, but are not virtualized
- provide lightweight process isolation (through cgroups) but share the Host OS kernel
- do not offer the same security boundaries

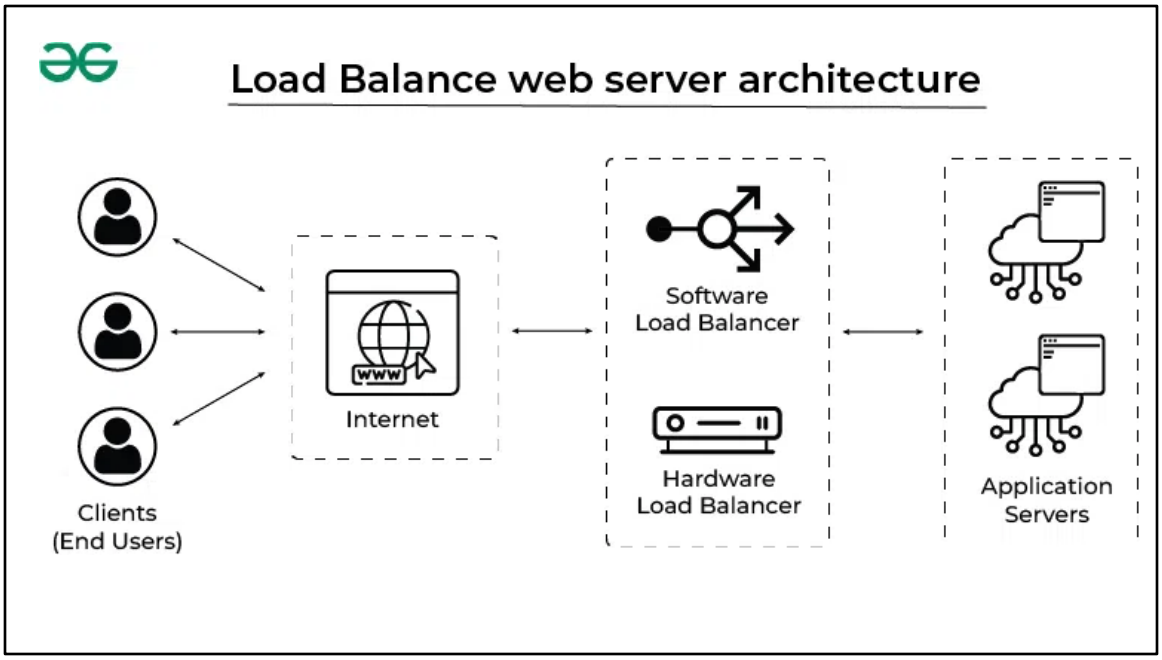
Web Server Architecture

Introduction to Web Server Architecture

- Web server architecture is the structure of a web server that allows it to handle multiple tasks efficiently. This involves:
- **How requests are managed**,
- **How the server resources are used**, and
- **How different parts of the server interact** to handle large numbers of client requests without slowing down.
- Web server architecture can be **single-process** (handling requests one at a time) or **multi-process** (handling multiple requests at the same time).

The web server architecture represents a web server's layout, structure, and components. It outlines the working system of a functional web server. This includes how the server handles incoming server requests and delivers responses to the client's computers.
There are two main approaches to web server architecture:
•Concurrent approach.
•Single-process-event-driven approach.


Single Server Architecture of Web Server
User Request Domain
Respond IP of Web Server
DNS Server
USER
HTTP Response with Data Files
Makes HTTP Request to IP
Web Server



1. Multi-process

A multi-process approach starts with a base process. It creates different single-threaded branch processes. These branches divide the requests among them. Once done, each branch handles a single request without affecting the other. This threading handles all requests without depending on other requests.

2. Multi-threaded

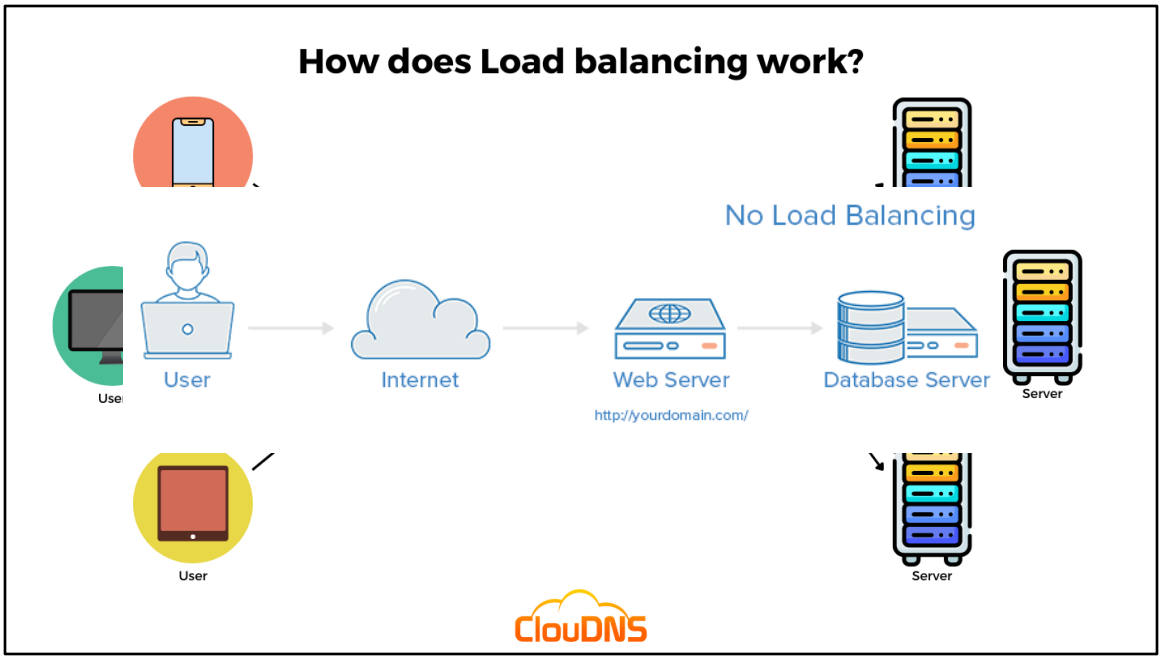
A server manages many client requests at once using multiple threads. The server generates a thread to interact with every client's request for other users simultaneously, making it more efficient for the server to respond quickly to many client requests.

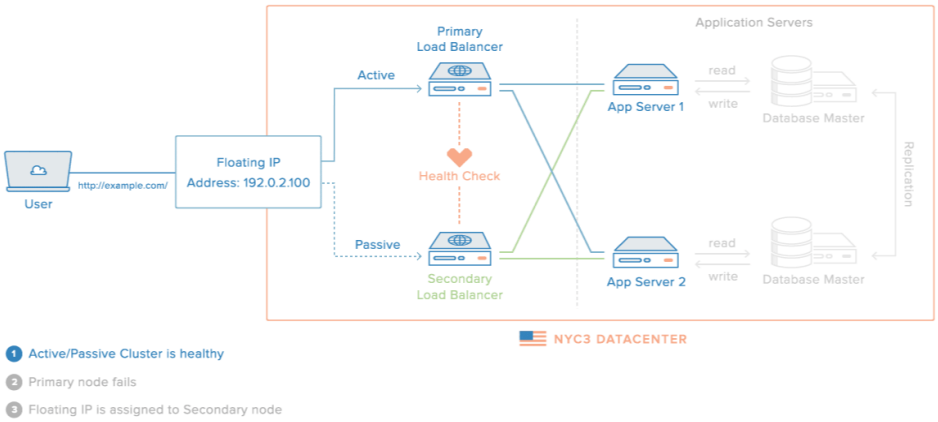
3. Hybrid

The hybrid method is a combination of both multi-process and multi-threaded methods where multiple processes are created. These multiple processes create threads that handle single requests at a time.

Load Balancing

- When multiple servers are used to spread out or balance the incoming requests so no single server gets overwhelmed.
- **Why Use Load Balancing?** When websites have many visitors, a single server can become overloaded. Load balancers distribute the requests to multiple servers, ensuring a smoother experience for users.
- **How It Works**: A load balancer sits between the client and the web servers. When a client requests a page, the load balancer directs it to the server with the least load.

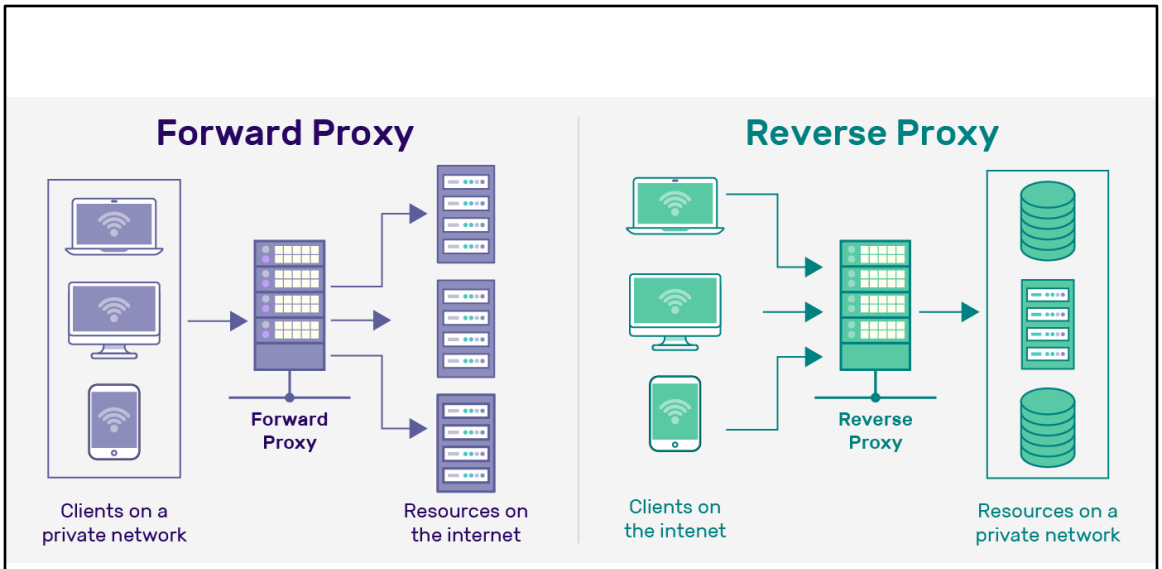

How does Load balancing work?
No Load Balancing
User
Internet
Web Server
http://yourdomain.com/
Database Server
Server
User
Server
ClouDNS



1. Active/Passive Cluster is healthy
2. Primary node fails
3. Floating IP is assigned to Secondary node

Reverse Proxy

- A **reverse proxy** is a type of server that sits in front of web servers and directs client requests to the appropriate backend server. It provides an extra layer of security and helps in load balancing as well.
- **Benefits**:
  - **Security**: Protects the identity and location of the backend server.
  - **Efficiency**: Redirects requests to different servers based on load.
- **Example**: NGINX can act as a reverse proxy. If a site has multiple servers, NGINX can manage which server a client is directed to.


Forward Proxy
Forward Proxy
Clients on a private network
Resources on the internet
Reverse Proxy
Reverse Proxy
Clients on the intenet
Resources on a private network

When to Use Which Web Server?

- Apache is highly flexible and configurable, making it an excellent choice for dynamic content and older web technologies.
- Nginx excels in performance, scalability, and handling high concurrency, making it ideal for serving static content, as a reverse proxy, or for modern applications.
- IIS is tightly integrated with the Windows Server ecosystem and is the go-to web server for enterprise level management

Apache

If you're building dynamic websites or web applications (especially with PHP, Python, or other server-side languages).You need flexibility with a modular architecture.
You require fine-grained configuration control through .htaccess files.
You are running on multiple operating systems (Linux, macOS, or Windows).

Nginx

If you're serving high-traffic websites and need a scalable, efficient solution.
You are primarily focused on static content and need a fast, lightweight web server.
You need to use reverse proxying or load balancing.
You're running a modern web application that requires efficient resource usage (e.g., microservices architecture).

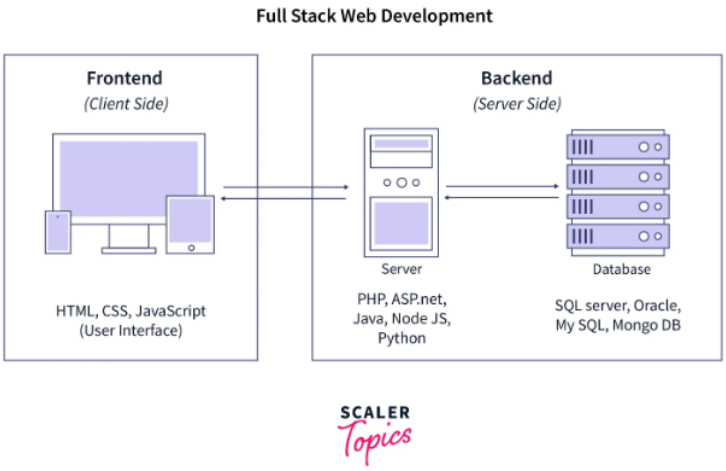
IIS

If You're working in a Windows environment and need deep integration with Microsoft technologies (e.g., ASP.NET, SQL Server).You're building applications using the .NET framework.
You need enterprise-level features with a GUI-based management interface.

Web Servers in Full Stack Development

What is Full Stack Development?

- Full stack development refers to building both the **front-end** (what the user sees) and **back-end** (how the website works) of a website or application.
- **Front-End**: This includes everything the user interacts with directly, such as HTML, CSS, and JavaScript.
- **Back-End**: This part involves the server, database, and application logic that make the website function.


Full Stack Web Development
Frontend
(Client Side)
HTML, CSS, JavaScript
(User Interface)
Backend
(Server Side)
Server
PHP, ASP.net,
Java, Node JS,
Python
Database
SQL server, Oracle,
My SQL, Mongo DB
SCALER
Topics

How Web Servers Fit Into Full Stack Development

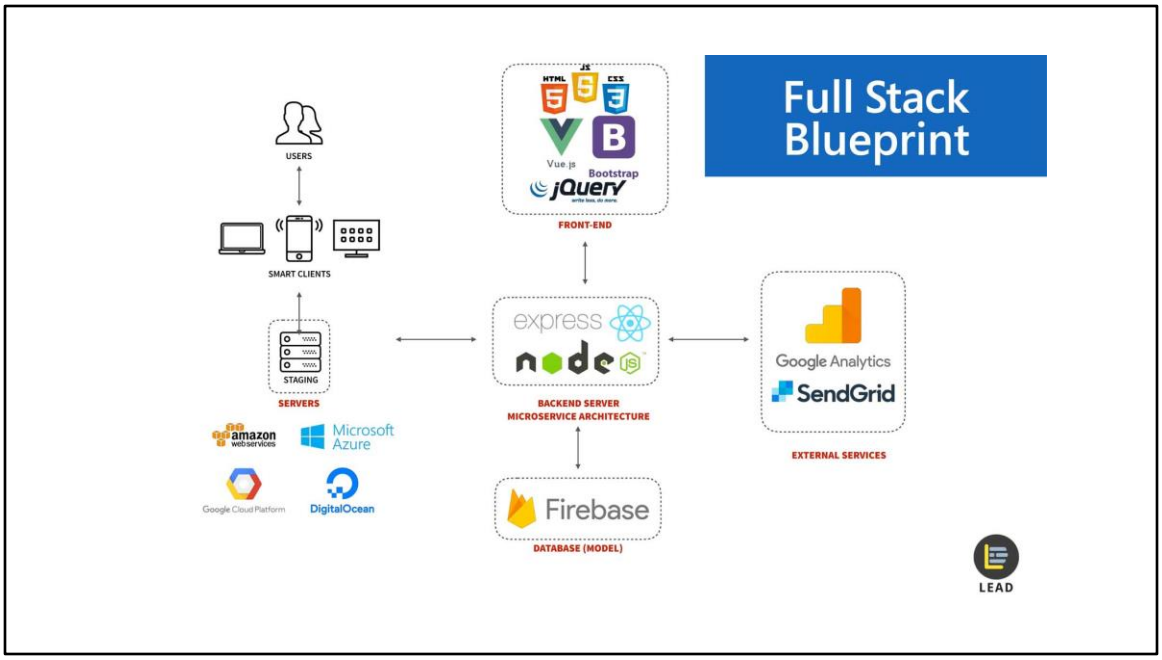
- **Connecting Front-End and Back-End**:
  - The web server is the **"middleman"** that connects the front-end and back-end.
  - When a user clicks a link, the request goes to the web server, which then retrieves the necessary data from the back-end and sends it back to the front-end for the user to see.
- **Handling Requests**:
  - Web servers help route user requests to the correct resources in the back-end, such as specific APIs or database queries.
  - This routing is essential in full stack development because it ensures that each part of the application communicates effectively.

How Web Servers Fit Into Full Stack Development

- **Routing**:
  - Servers handle the "routing" of requests, which means directing different requests to different parts of the application.
  - For example, if a user wants to log in, the web server routes the request to the login function in the back-end.

A typical e-commerce site is an example of Full Stack Development

- **Front-End**: Displays products, images, and buttons.
- **Back-End**: Manages product inventory, processes payments.
- **Web Server's Role**: Ensures that the user sees the updated stock levels by fetching data from the database.


Full Stack Blueprint
USERS
SMART CLIENTS
STAGING
SERVERS
amazon webservices
Microsoft Azure
Google Cloud Platform
DigitalOcean
HTML
CSS
JS
Vue.js
Bootstrap
jQuery
FRONT-END
express
node
BACKEND SERVER
MICROSERVICE ARCHITECTURE
Firebase
DATABASE (MODEL)
Google Analytics
SendGrid
EXTERNAL SERVICES
LEAD

Relationship to Full Stack Development

Front-end vs. Back-end:

- **Front-end**: HTML, CSS, JavaScript - runs on client's browser.
- **Back-end**: Processes client requests, communicates with databases, generates responses.

Role of Web Servers:

- **Handling Requests**: Connects front-end and back-end by serving files and data.
- **Routing**: Maps requests to corresponding back-end functions.
- **Example in Full Stack Architecture**: Diagram showing front-end, back-end, and web server relationships.

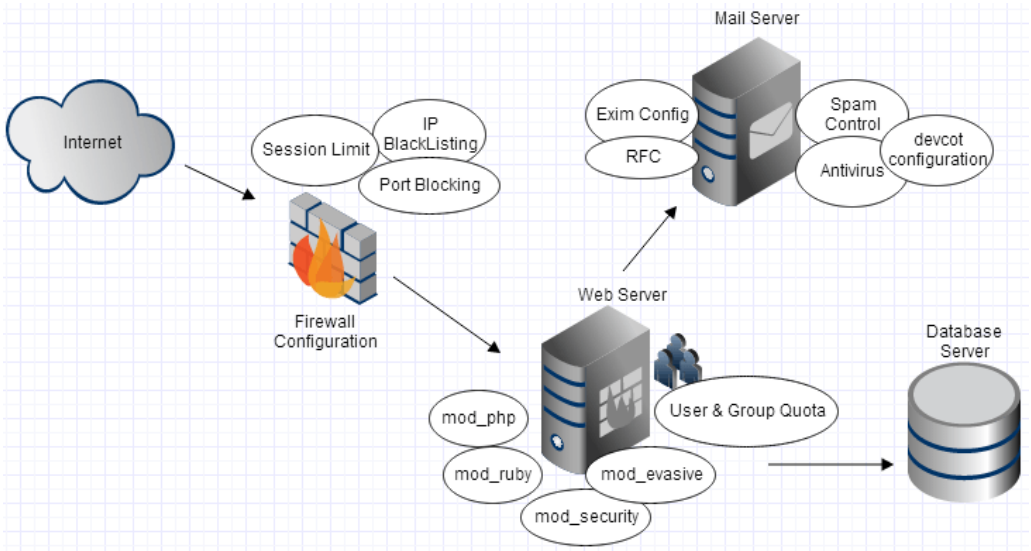
Basic Server Configuration

Server Setup:

- Installation of Apache, NGINX, or IIS.
- Setting up directory structure, permissions.

Configuring HTTP/HTTPS:

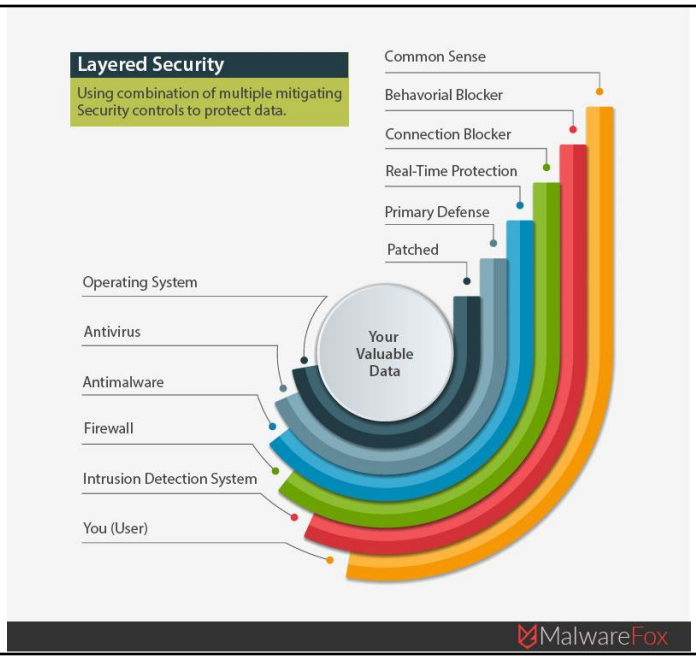
- Enabling SSL/TLS.
- Configuring HTTP headers for security.

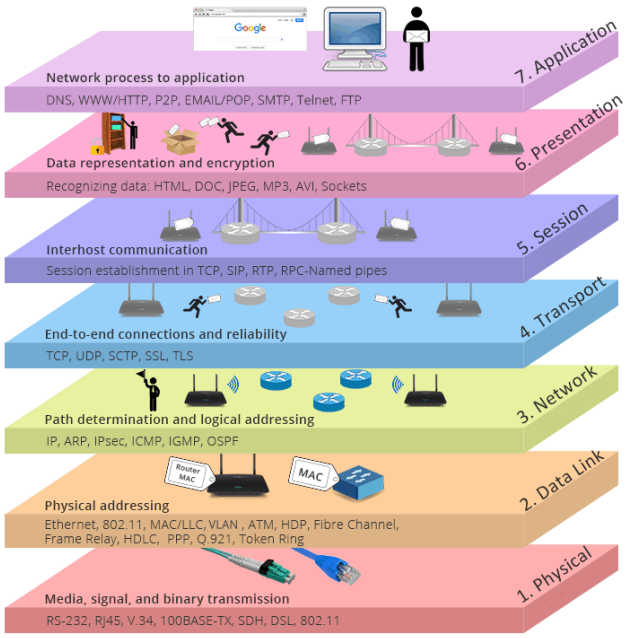


https://www.guru99.com/apache.html

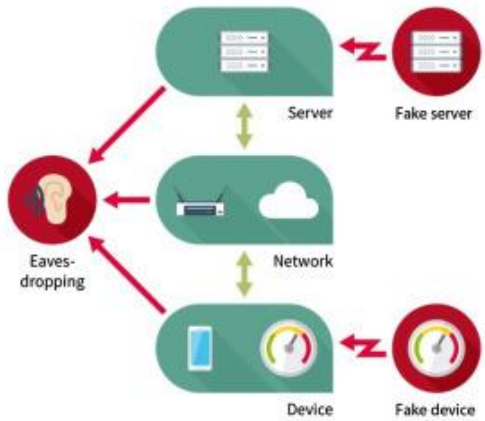
Server Security Essentials

- **Firewall Configurations**: Controlling access to the server.
- **Authentication and Authorization**: Securing endpoints with user roles and permissions.
- **SSL/TLS Certificates**: Encrypting data in transit.
- **Hardening**: Regular updates, disabling unused modules.


Layered Security
Using combination of multiple mitigating Security controls to protect data.
Common Sense
Behavorial Blocker
Connection Blocker
Real-Time Protection
Primary Defense
Patched
Operating System
Antivirus
Antimalware
Firewall
Intrusion Detection System
You (User)
Your Valuable Data
MalwareFox


Google
7. Application
Network process to application
DNS, WWW/HTTP, P2P, EMAIL/POP, SMTP, Telnet, FTP
6. Presentation
Data representation and encryption
Recognizing data: HTML, DOC, JPEG, MP3, AVI, Sockets
5. Session
Interhost communication
Session establishment in TCP, SIP, RTP, RPC-Named pipes
4. Transport
End-to-end connections and reliability
TCP, UDP, SCTP, SSL, TLS
3. Network
Path determination and logical addressing
IP, ARP, IPsec, ICMP, IGMP, OSPF
Router MAC
MAC
2. Data Link
Physical addressing
Ethernet, 802.11, MAC/LLC, VLAN , ATM, HDP, Fibre Channel,
Frame Relay, HDLC, PPP, Q.921, Token Ring
1. Physical
Media, signal, and binary transmission
RS-232, RJ45, V.34, 100BASE-TX, SDH, DSL, 802.11

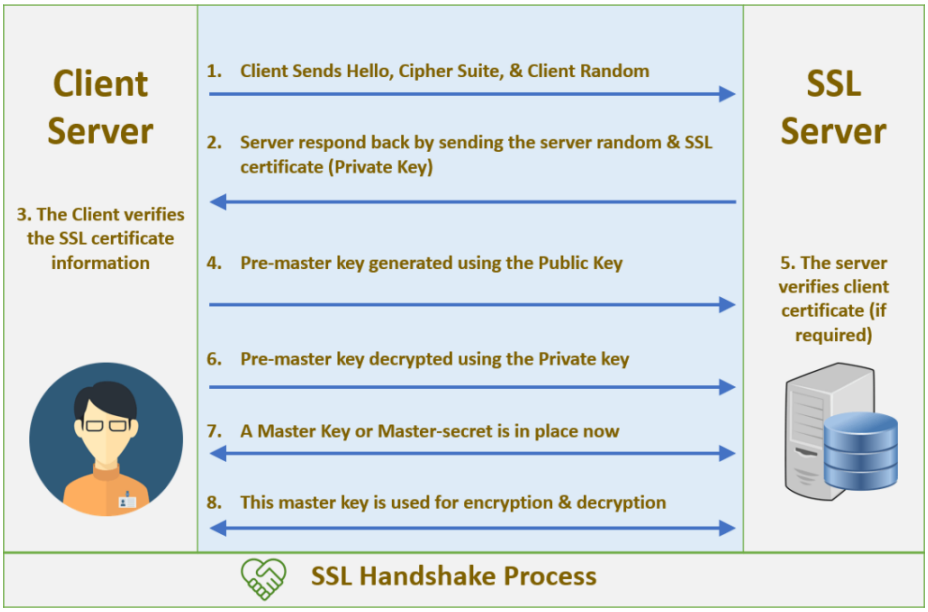
An **Eavesdropper** listening in on data or commands can reveal confidential information about the operation of the infrastructure.



A **Fake server** sending incorrect commands can be used to trigger un-planned events, to send some physical resource (water, oil, electricity, etc.) to an unplanned destination, and so forth.

A **Fake device** injecting fake measurements can disrupt the control processes and cause them to react inappropriately or dangerously, or can be used to mask physical attacks.*

*example for BAD USB.


Client Server
SSL Server
1. Client Sends Hello, Cipher Suite, & Client Random
2. Server respond back by sending the server random & SSL certificate (Private Key)
3. The Client verifies the SSL certificate information
4. Pre-master key generated using the Public Key
5. The server verifies client certificate (if required)
6. Pre-master key decrypted using the Private key
7. A Master Key or Master-secret is in place now
8. This master key is used for encryption & decryption
SSL Handshake Process

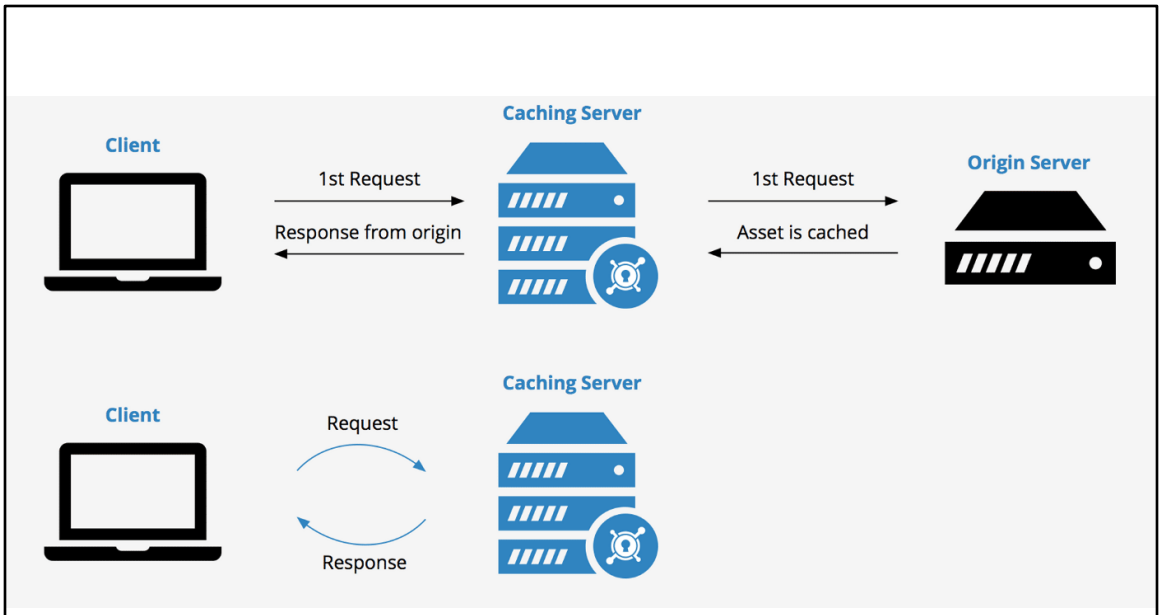
Advanced Server Concepts: Caching

- **Server Caching** Temporarily stores web files and data on the server for later use, effectively reducing the server's load and latency. Use of Varnish, Redis to speed up response times
- **Varnish** is an HTTP accelerator and caching reverse proxy that improves website performance by caching static content by storing pre-rendered HTML pages in memory; especially useful for handling heavy traffic
- **Redis** is an open-source in-memory data store that can be used as a database, cache, or message broker; efficient for caching dynamic content in web applications. Redis is also used for session storage and other data-level caching

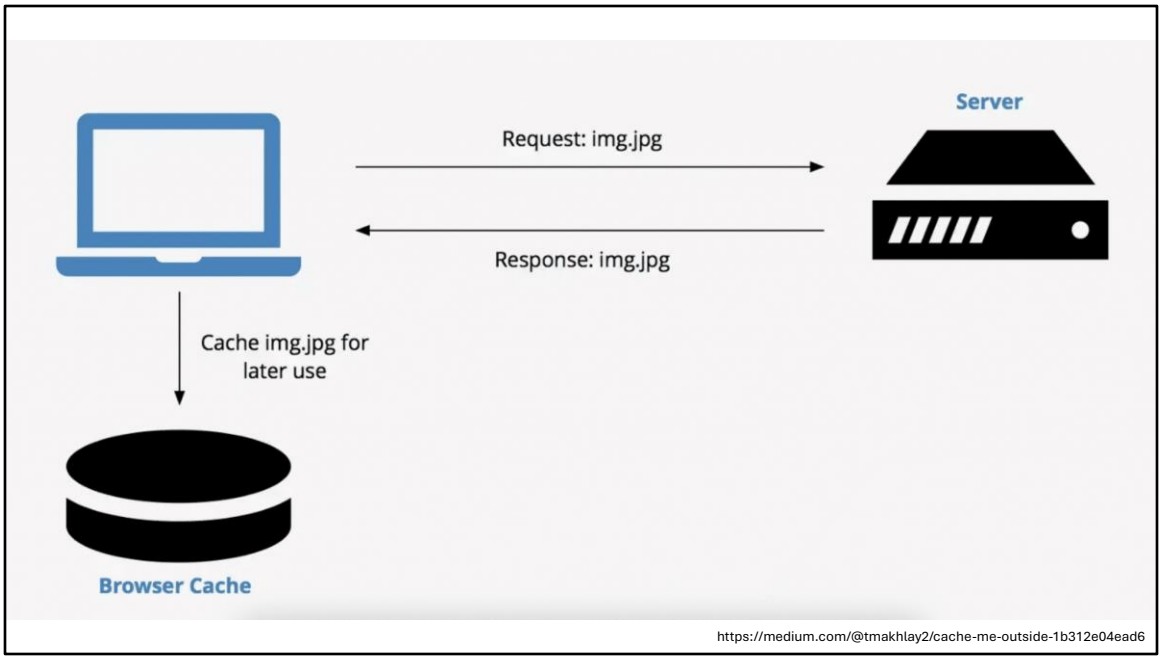
Server Side Caching:
1. When the user visits a website and requests the web page, the website retrieves data from the server, generates that webpage, and displays it to the user.
2. After the response has been sent back to the user, the server creates a copy of the webpage and stores it as a cache.
3. The next time a user revisits the website, instead of going through the whole process again, it displays the already saved (or cached) copy of the webpage – thus making the website load faster!

Varnish is an HTTP accelerator and caching reverse proxy that improves website performance by caching static content. Varnish stores pre-rendered HTML pages in memory, which reduces the load on web servers and improves user experience. Varnish is especially useful for handling heavy traffic

Redis is an open-source in-memory data store that can be used as a database, cache, or message broker. Redis is primarily used as an object cache backend, and is efficient for caching dynamic content in web applications. Redis is also used for session storage and other data-level caching



https://medium.com/@tmakhlay2/cache-me-outside-1b312e04ead6



Client-side caching or browser caching, temporarily stores the copy of a webpage in the browser's memory (a folder created by the browser).
So, the next time a user revisits a website with client-side caching enabled, it won't make a call to the server for the data. Instead, it will fetch it from the browser cache folder located on the user's device.

Advanced Server Concepts

Session Management: Tracking user sessions, often necessary for dynamic websites.

Load Balancing and Failover:

- Techniques like round-robin, IP hashing.
- Ensuring high availability with multiple server nodes.

Usage of more than one tool combined: Varnish and Redis can be used together to optimize performance. In Magento, Redis handles session storage and other data-level caching, while Varnish focuses on full-page caching.

Round robin and IP hashing are both load balancing methods that distribute incoming traffic to servers:

•**Round robin**

•Sequentially distributes traffic to each server in a list, repeating the list after each server receives a connection. This method is suitable for servers with identical specifications.

•**IP hashing**

•Uses the source IP address of an incoming request to route traffic to a specific backend server. This method ensures that requests from the same client are consistently routed to the same server.

Magento is an open-source e-commerce platform written in PHP.
Magento source code is distributed under Open Software License

Server Performance Optimization

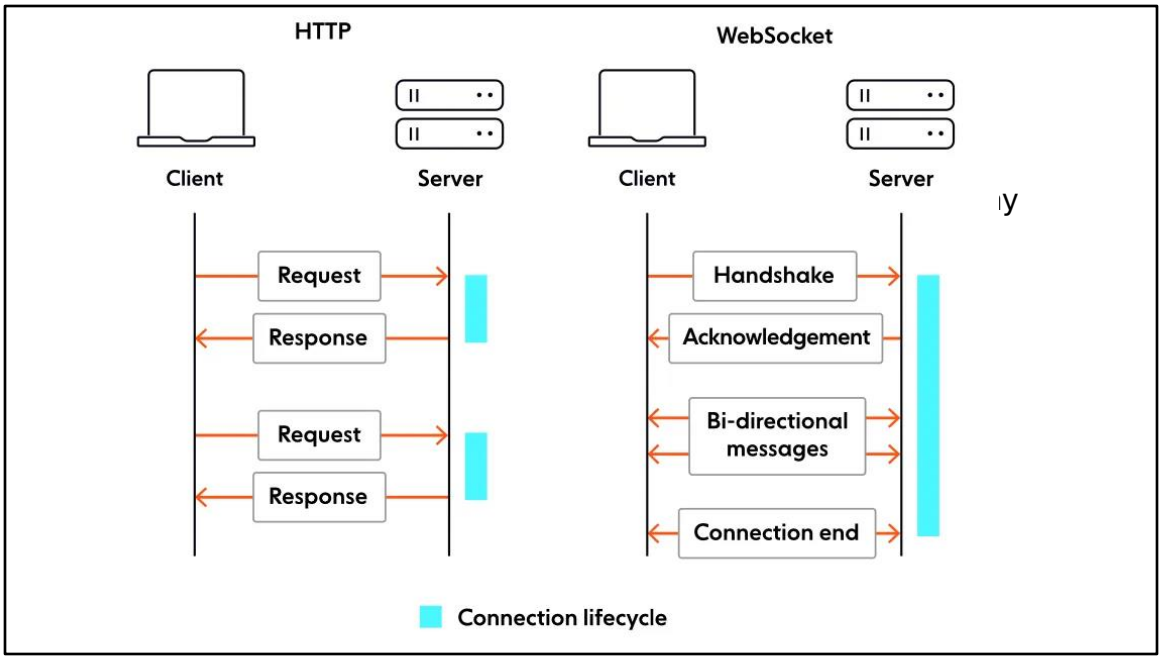
Connection Handling:

- Persistent connections (HTTP/1.1 Keep-Alive).
- Connection pooling and limiting.

Compression: Using Gzip to reduce file sizes.

Asynchronous Processing:

- Server push, WebSockets for real-time data.
- Asynchronous handling is the key in dynamic servers.


HTTP
WebSocket
Client
Server
Client
Server
Request
Response
Request
Response
Handshake
Acknowledgement
Bi-directional messages
Connection end
Connection lifecycle

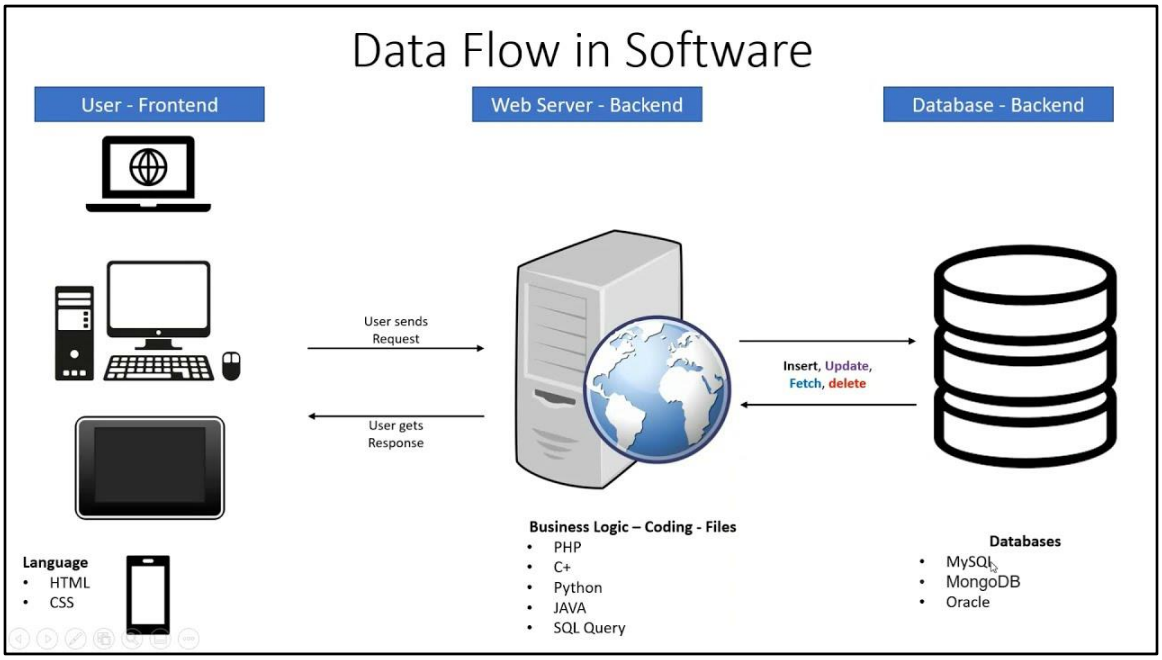
Database Interaction with Web Servers

Database as Back-end Component:

- Relational (e.g., MySQL, PostgreSQL) vs. Non-relational (e.g., MongoDB).
- Connecting databases to web servers through ORM or direct connections.
- ORM frameworks, or Object Relational Mapper frameworks, are tools that help developers interact with databases using object-oriented programming (OOP) languages

Database Caching: Using database caching to reduce response times.

ORMs translate between the data representations used by databases and those used in OOP. They create a layer between the programming language and a relational database using metadata descriptors.


Data Flow in Software
User - Frontend
Web Server - Backend
Database - Backend
User sends Request
User gets Response
Insert, Update, Fetch, delete
Language
HTML
CSS
Business Logic – Coding - Files
PHP
C+
Python
JAVA
SQL Query
Databases
MySQL
MongoDB
Oracle

Cloud Servers

- Web hosting and server management have moved to virtual environments, providing significant advantages such as scalability, flexibility, cost-efficiency, and global accessibility.
- A Cloud Web Server is essentially a virtual server hosted on a cloud computing infrastructure, capable of running web applications, websites, databases, and more.
- These servers allow **businesses** and **developers** to host and manage their websites/applications without needing to own or manage physical hardware.
- Cloud web servers are primarily delivered using three common models: Infrastructure as a Service **(IaaS)**, Platform as a Service **(PaaS)**, and Software as a Service **(SaaS)**.

Cloud computing has revolutionized the way we deploy, manage, and scale web applications. Traditionally, websites and web applications were hosted on physical servers located in data centers. However, with the advent of cloud computing, web hosting and server management have moved to virtual environments, providing significant advantages such as scalability, flexibility, cost-efficiency, and global accessibility.
A Cloud Web Server is essentially a virtual server hosted on a cloud computing infrastructure, capable of running web applications, websites, databases, and more. These servers allow businesses and developers to host and manage their websites/applications without needing to own or manage physical hardware.
Cloud web servers are primarily delivered using three common models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS).
Let's delve deeper into these concepts.

Infrastructure as a Service (IaaS)

- Most fundamental cloud service model.
- Infrastructure components like servers, storage, networking, and virtual machines (VMs) on a pay-as-you-go basis.
- It abstracts the underlying physical hardware and gives developers more control over the operating system and software installed on their virtual machines.
- **Example providers:** Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP)
- **Components:** Virtual Machines (VMs), storage (block or object storage), networking (load balancers, DNS), and security features like firewalls.
- **Use Cases:** Hosting websites, running applications, backup and disaster recovery, big data analytics.
- Scalable, Cost-Efficient, Customizable
- Requires technical knowledge

Most fundamental cloud service model. It provides virtualized computing resources over the internet. IaaS offers infrastructure components like servers, storage, networking, and virtual machines (VMs) on a pay-as-you-go basis. It abstracts the underlying physical hardware and gives developers more control over the operating system and software installed on their virtual machines.

- Example providers: Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP)
- Components: Virtual Machines (VMs), storage (block or object storage), networking (load balancers, DNS), and security features like firewalls.
- Use Cases: Hosting websites, running applications, backup and disaster recovery, big data analytics.

Advantages:

- Scalability: Easily scale resources up or down depending on demand.
- Cost-efficiency: Pay only for what you use.
- Flexibility: Customizable, as you have control over the operating system

and software.

Challenges:

- You are responsible for managing the operating system and software updates.
- Requires technical knowledge to configure and maintain the infrastructure.

Platform as a Service (PaaS)

- Abstracts much of the underlying infrastructure management; developers can use to build, deploy, and scale applications without worrying about the underlying system
- PaaS typically provides **pre-configured development environments**, middleware, databases, and runtime environments. Developers can focus purely on writing and deploying their code.
- **Example providers:** Heroku, Google App Engine, Microsoft Azure App Services, AWS Elastic Beanstalk.
- **Components:** Development frameworks, application hosting environments, databases, monitoring tools.
- **Use Cases:** Web app development, mobile app backends, microservices architectures.
- Rapid development and deployment, no infrastructure management, support multiple frameworks and languages
- Limited control over underlying infrastructure

PaaS abstracts much of the underlying infrastructure management and focuses more on providing a platform that developers can use to build, deploy, and scale applications without worrying about the underlying hardware, operating system, or networking.
PaaS typically provides pre-configured development environments, middleware, databases, and runtime environments. Developers can focus purely on writing and deploying their code.
Example providers: Heroku, Google App Engine, Microsoft Azure App Services, AWS Elastic Beanstalk.
Components: Development frameworks, application hosting environments, databases, monitoring tools.
Use Cases: Web app development, mobile app backends, microservices architectures.

Advantages:

- Rapid development and deployment: Pre-configured environments streamline development.
- No infrastructure management: The cloud provider handles infrastructure maintenance.

- Supports multiple languages and frameworks.

Challenges:

- Limited control over the underlying infrastructure.
- Less flexibility compared to IaaS in terms of customization.
- Potential vendor lock-in, as you might become reliant on specific PaaS services.

Software as a Service (SaaS)

- SaaS provides fully managed software applications over the internet.
- **Example providers:** Google Workspace (formerly G Suite), Microsoft 365, Dropbox, Salesforce.
- **Components:** Fully managed applications (e.g., email services, CRM software).
- **Use Cases:** Enterprise software, email, file storage, collaboration tools.
- No installation , Easily accessible, No need for technical knowledge to use the software.
- Limited customization options, dependencies, data privacy and security concerns in multi-tenant environments.

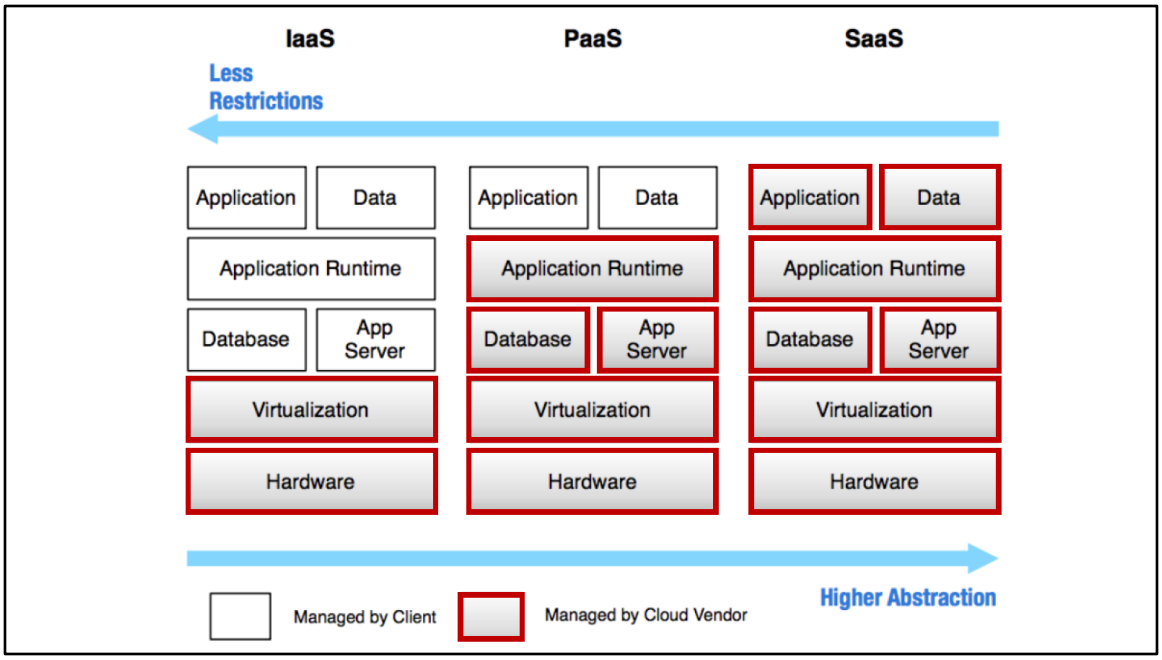
In this model, the cloud provider takes care of all aspects of the software, including infrastructure, security, updates, and maintenance.

Advantages:

- No installation or maintenance required by the user.
- Easily accessible from anywhere with internet access.
- No need for technical knowledge to use the software.

Challenges:

- Limited customization options.
- Dependence on the provider for updates and changes.
- Data privacy and security concerns in multi-tenant environments.


IaaS
PaaS
SaaS
Less Restrictions
Application
Data
Application Runtime
Database
App Server
Virtualization
Hardware
Higher Abstraction
Managed by Client
Managed by Cloud Vendor

- Infrastructure as a service (IaaS) [***E.g.*** AWS EC2, Google Compute Engine]
  - API-driven infrastructure (web servers) at scale
  - Provides the ability to write a script that automates retrieving new (virtual) server capacity
- Platform as a Service (PaaS) [***E.g.*** Heroku, CloudFoundry, App Engine]
  - Managed application runtimes (e.g., web servers) that are built on top of IaaS for scalability
  - Underlying infrastructure (server) is abstracted away, configuration can provide directives
    - Deploy web applications by providing directives on process to start or providing container
- Software as a Service (SaaS) [***E.g.*** Salesforce, Office 365, Adobe Creative Cloud]
  - Delivers software applications over the internet, eliminating the need for users to install, maintain, and manage the software locally.
  - Typically offered on a subscription basis, allowing users to pay for the software on a recurring basis
- Function as a Service (FaaS) [***E.g.*** AWS Lambda, Azure Functions, Google Cloud Functions]
  - **aka Serverless computing** - allows developers to execute individual functions or pieces of code in response to events without managing the underlying infrastructure
  - The cloud provider automatically scales and allocates resources as needed

- Container as a Service (CaaS) [***E.g.*** Amzon: EKS/ ECS, GCP: GKE/ Run, Azure: AKS, ACI]
  - Enables developers to deploy, manage, and scale containerized applications without managing the underlying infrastructure
- Database as a Service (DBaaS) [***E.g.*** Amazon RDS, Google Cloud SQL, Azure Database]
  - Provides managed database services without the need for users to install, configure, and manage databases
- Backend as a Service (BaaS) [***E.g.*** Firebase, AWS Amplify, Kinvey]
  - Offers pre-built backend services for mobile and web applications
  - Includes features like user authentication, databases, and cloud storage
- Desktop as a Service (DaaS) [***E.g.*** Amazon WorkSpaces, Azure Virtual Desktop, VMware Horizon Cloud]
  - Delivers virtual desktops hosted on a cloud infrastructure, enabling remote access to desktop environments
  - Provides flexibility and accessibility for users, reducing the need for physical desktop hardware
- Identity as a Service (IDaaS) [***E.g.*** Okta, Azure Active Directory, Auth0]
  - Offers identity and access management services, providing authentication & authorization services
  - Enables organizations to manage user identities securely and ensure appropriate access controls

Uses of Web Server

Web servers have a wide range of uses and applications, as they are the backbone of the internet and the means by which web resources are delivered to users. Here are some of the most common uses of web servers:

1. Hosting websites: Web servers are used to host websites, making them accessible to users around the world. They serve up HTML, CSS, JavaScript, and other web resources that make up web pages.
2. Running web applications: Web servers are used to host web applications that run in the browser, such as online shopping carts, social media platforms, and productivity tools.
3. Streaming media: Web servers are used to stream audio and video content over the internet, allowing users to access music, movies, and other multimedia resources.
4. Managing online databases: Web servers are used to manage online databases, allowing users to access and manipulate data via web-based interfaces.
5. Cloud computing: Web servers are used to power cloud computing services, allowing users to access computing resources and data storage via the Internet.
6. Internet of Things (IoT): Web servers are used to power IoT devices, allowing them to communicate and exchange data over the Internet.
7. File sharing: Web servers are used to share files over the internet, allowing users to access and download files such as documents, images, and videos.

What is Web Server Security?

1. **Regular software updates:** It's important to keep the web server software and applications up-to-date to address any security vulnerabilities or bugs.
2. **Secure configuration:** The web server should be configured with strong security settings, such as secure file permissions, SSL/TLS encryption, and secure authentication mechanisms.
3. **Access control:** The web server should be configured to restrict access to sensitive data and resources, including password-protecting directories and using firewall rules.
4. **Security testing:** Regular security testing and vulnerability scanning should be performed to identify and address any potential security threats.
5. **Monitoring and logging:** Monitoring the web server logs and activity can help detect and respond to security incidents in a timely manner.
6. **Backup and recovery:** Regular backups of the web server and data should be performed to ensure quick recovery in case of data loss or other security incidents.

Web server security refers to the measures and practices used to protect a web server from unauthorized access, malicious attacks, and other security threats. A secure web server ensures that sensitive data and resources are protected from unauthorized access, ensuring the confidentiality, integrity, and availability of web resources.
Here are some common web server security measures:
1.Regular software updates: It's important to keep the web server software and applications up-to-date to address any security vulnerabilities or bugs.
2.Secure configuration: The web server should be configured with strong security settings, such as secure file permissions, SSL/TLS encryption, and secure authentication mechanisms.
3.Access control: The web server should be configured to restrict access to sensitive data and resources, including password-protecting directories and using firewall rules.
4.Security testing: Regular security testing and vulnerability scanning should be performed to identify and address any potential security threats.
5.Monitoring and logging: Monitoring the web server logs and activity can help detect and respond to security incidents in a timely manner.
6.Backup and recovery: Regular backups of the web server and data should be performed to ensure quick recovery in case of data loss or other security incidents.

Additional Resources

- **MDN Web Docs**: https://developer.mozilla.org/
  - Excellent resource for visual explanations on web servers and protocols.
- **DigitalOcean Tutorials**: https://www.digitalocean.com/community/tutorials
  - Provides step-by-step guides and diagrams on server configuration and deployment.
- **Apache and NGINX Documentation**:
  - Apache: https://httpd.apache.org/docs/
  - NGINX: https://nginx.org/en/docs/
- **Cloud Providers' Free Tier**: Check cloud providers like AWS, Google Cloud for trial accounts to practice deployment.
- https://www.digitalocean.com/community/tutorials/how-to-configure-nginx-as-a-web-server-and-reverse-proxy-for-apache-on-one-ubuntu-18-04-server